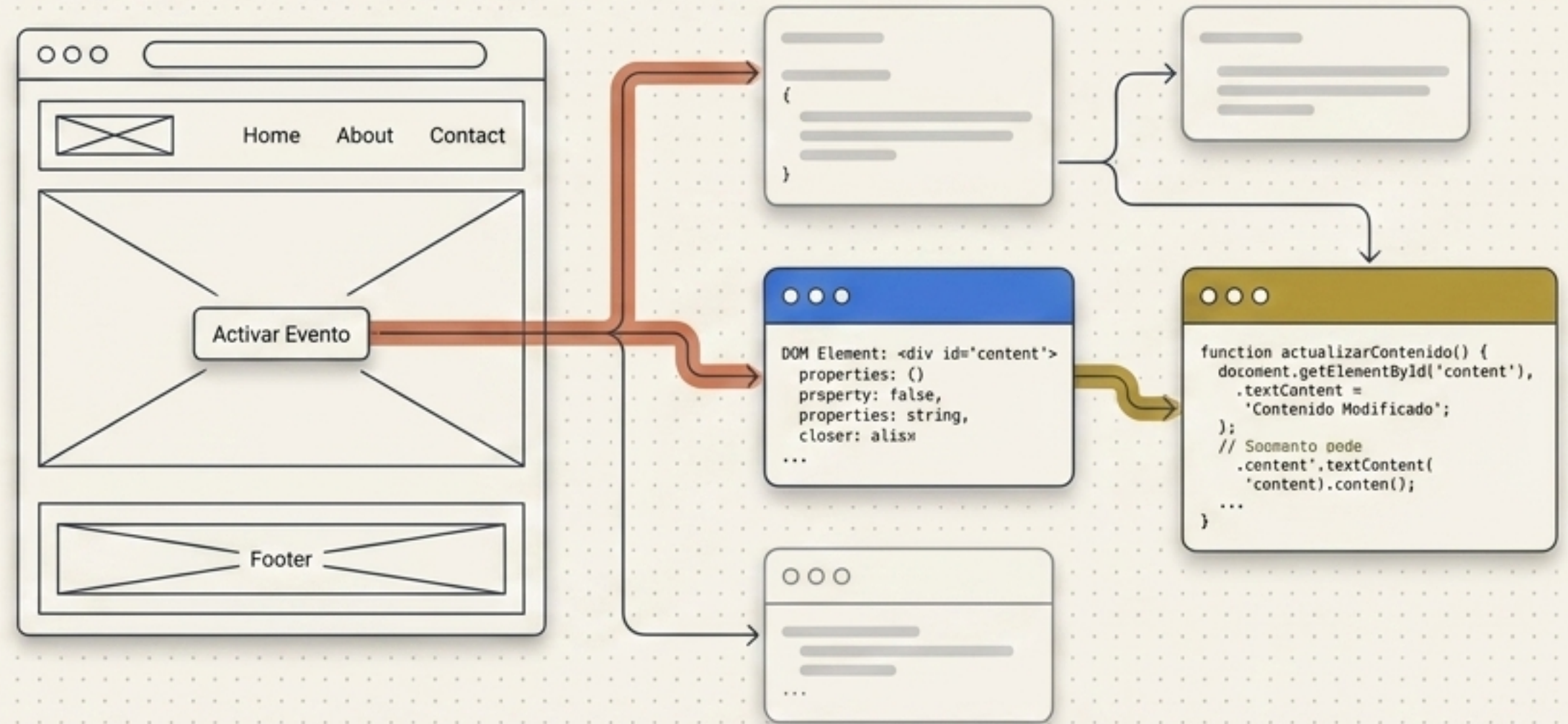


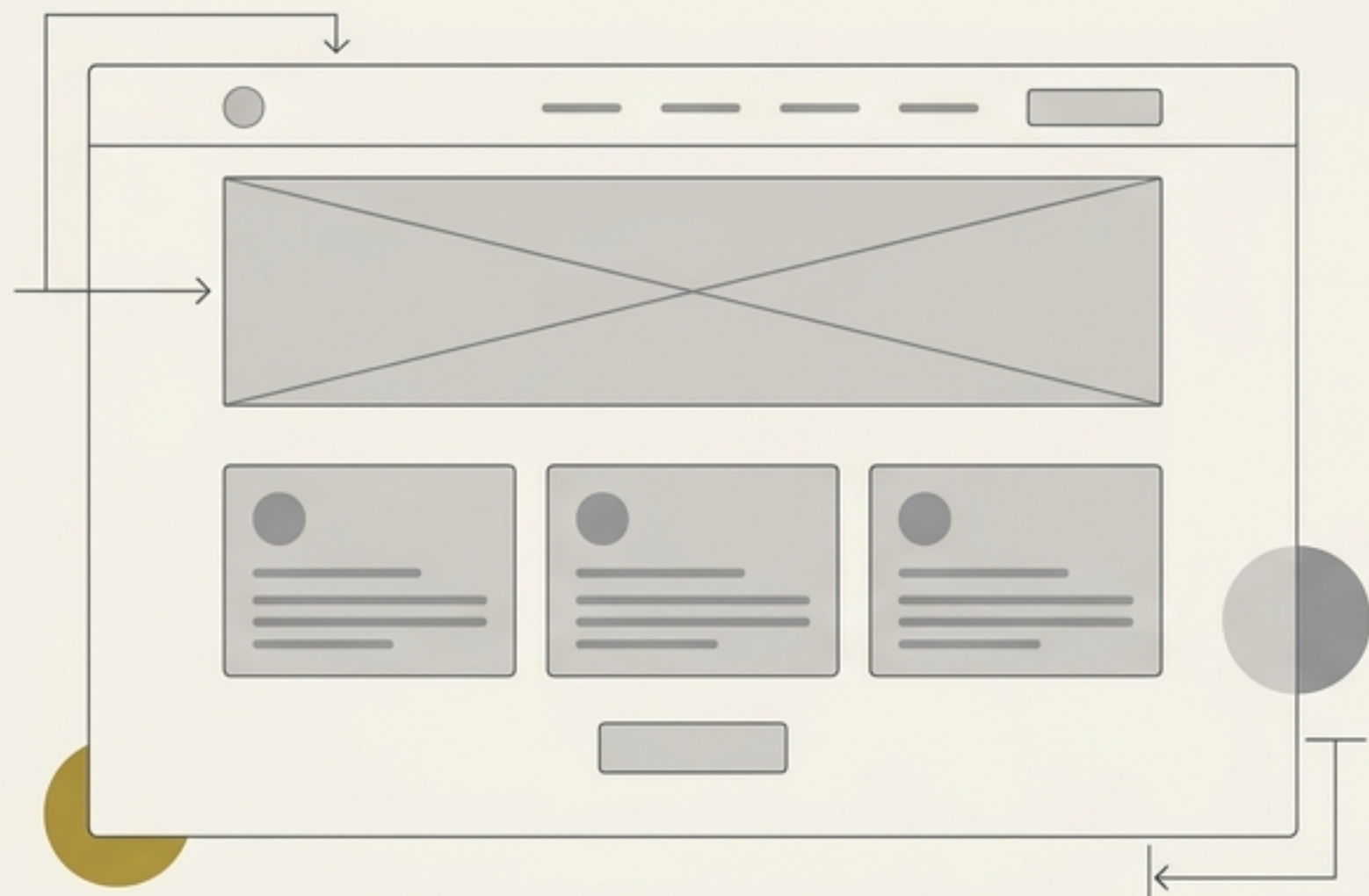
# El Circuito Interactivo

Modificando el contenido de  
una página con JavaScript

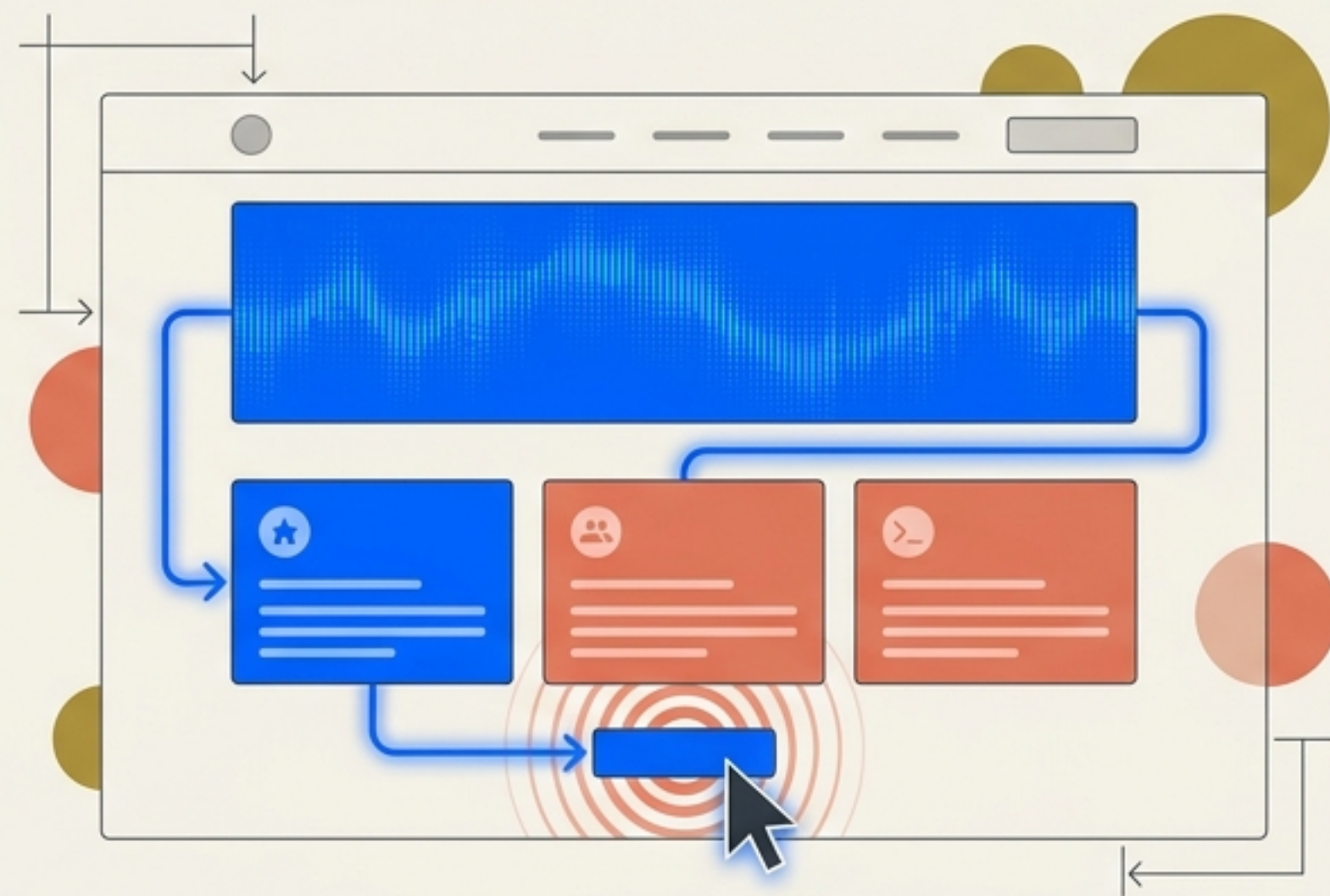


# ¿Qué son los eventos?

Un evento es una acción o suceso en el navegador que el código JavaScript puede detectar y manejar. Son los **impulsos** que permiten que una aplicación responda dinámicamente a las acciones del usuario.

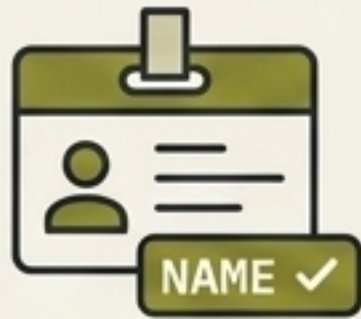


Sin Eventos: Un motor apagado



Con Eventos: Un circuito interactivo

# Fundamentos del Código



## Función Nombrada

Función con un identificador claro tras la palabra clave function. Ideal para reutilizar.

```
function suma(a, b) {}
```



## Función Anónima

Definida sin nombre directamente en el lugar donde se necesita ejecutar.

```
const resta = function(a, b) {}
```

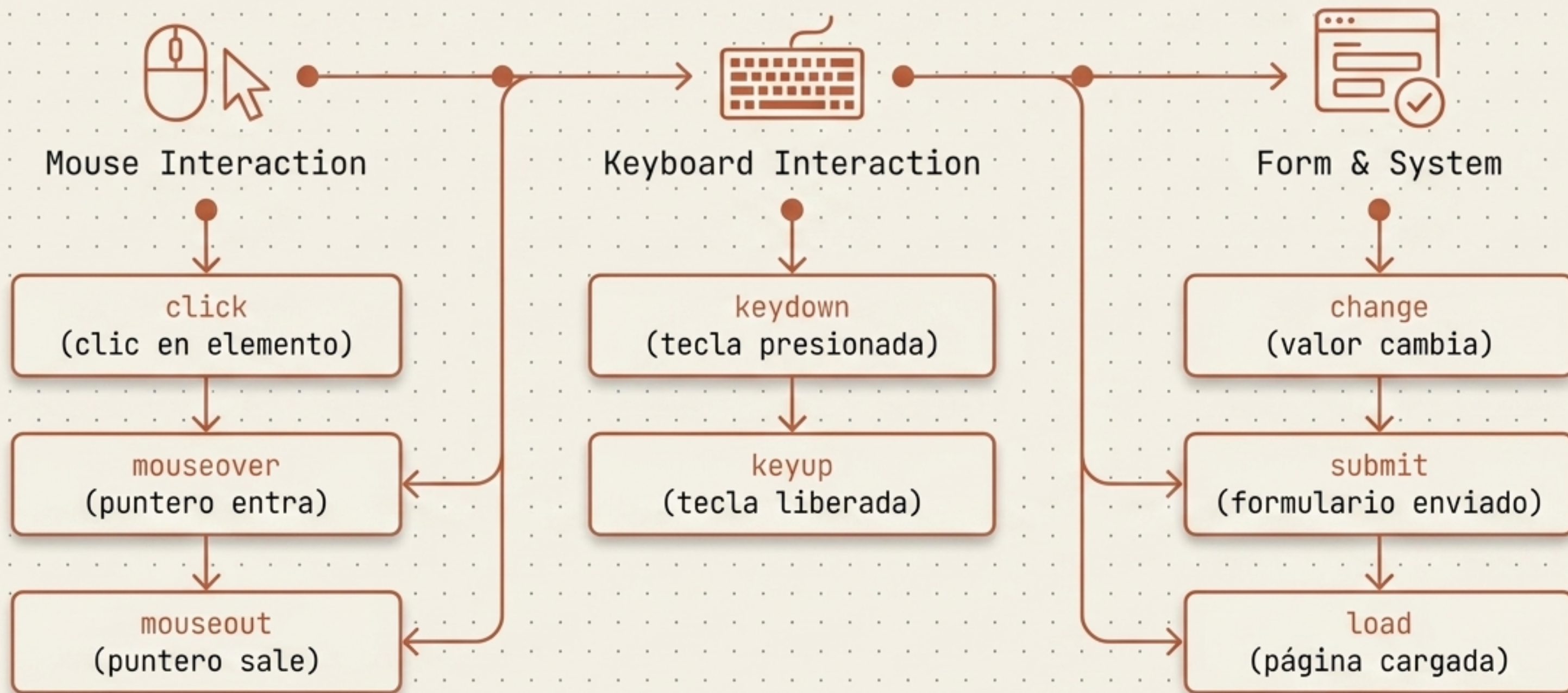


## Declaración

Instrucción que realiza una acción específica o define una entidad estructurada en el código.

# Taxonomía de Eventos Comunes

Las acciones del usuario y del sistema actúan como interruptores que disparan la ejecución del código.



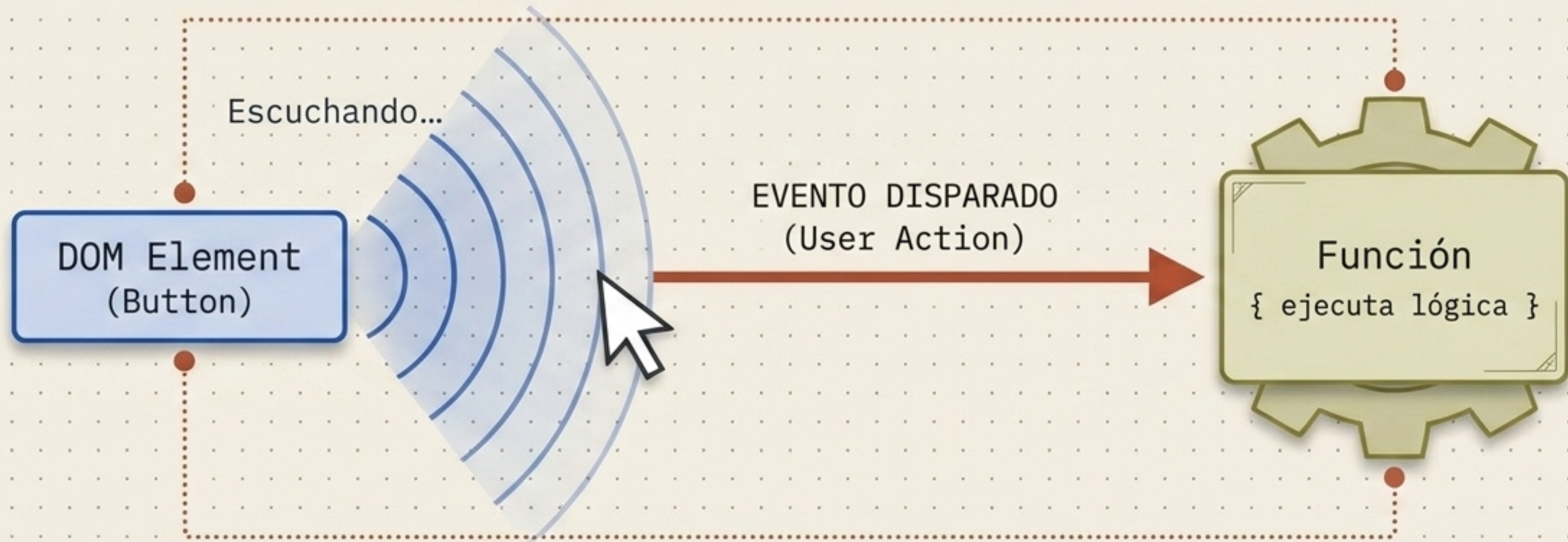
# Matriz de Equivalencias: La Evolución de la Sintaxis

Para separar la estructura HTML del comportamiento JavaScript, la programación moderna elimina el prefijo 'on' y traslada la responsabilidad a los Event Listeners.

| Legacy HTML Atributos                 | JavaScript Moderno |
|---------------------------------------|--------------------|
| <del>&lt;button onclick=...&gt;</del> | → 'click'          |
| <del>&lt;input onchange=...&gt;</del> | → 'change'         |
| <del>&lt;form onsubmit=...&gt;</del>  | → 'submit'         |
| <del>&lt;body onload=...&gt;</del>    | → 'load'           |

# El Metáfora del Oyente (Event Listener)

Un Event Listener es un observador asignado a un elemento del DOM que se queda a la espera, como un radar, de un suceso. Cuando el evento entra en su rango, dispara la función asociada.



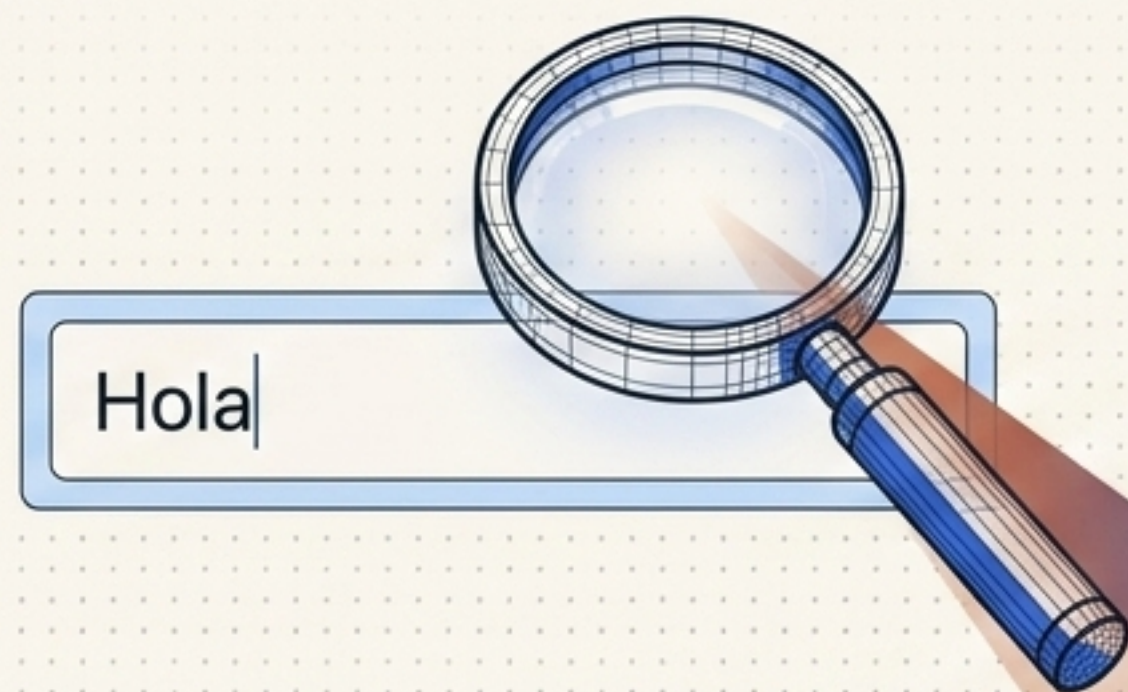
# Anatomía de addEventListener

El método principal para conectar el mundo estático del HTML con el mundo dinámico de JavaScript.



# La Lupa de Inspección: Leyendo Datos

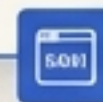
La función del listener recibe un parámetro event. Este objeto contiene toda la información del suceso, permitiéndonos inspeccionar con precisión qué interactuó el usuario.



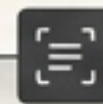
`event` (El objeto global del suceso)



`.target` (El elemento HTML específico: el input)



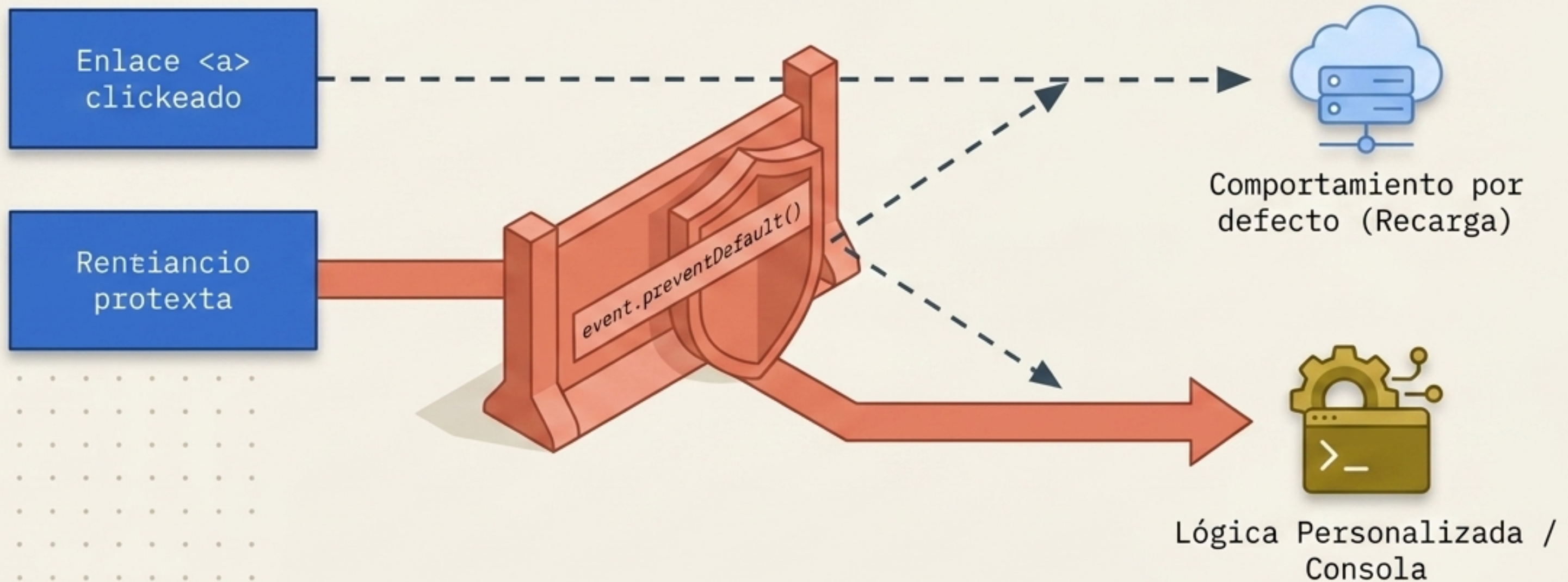
`.value` (El texto extraído: 'Hola')



```
console.log(event.target.value);
```

# El Escudo Protector: `preventDefault()`

Algunos elementos tienen comportamientos predeterminados (como un enlace que recarga la página). `preventDefault()` bloquea esa acción automática, permitiéndote tomar el control total del flujo.



# Matriz Arquitectónica: ¿Anónima o Nombrada?

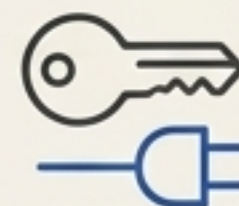
La forma en que declaras la función determina tu control futuro sobre el evento.



## Función Anónima

```
addEventListener('click', function() {  
  ...  
})
```

- Rápida para acciones únicas de un solo uso.
- **Imposible** de referenciar nuevamente en el futuro.
- **No** se puede eliminar con `removeEventListener`.



## Función Nombrada

```
function accion() {...}
```

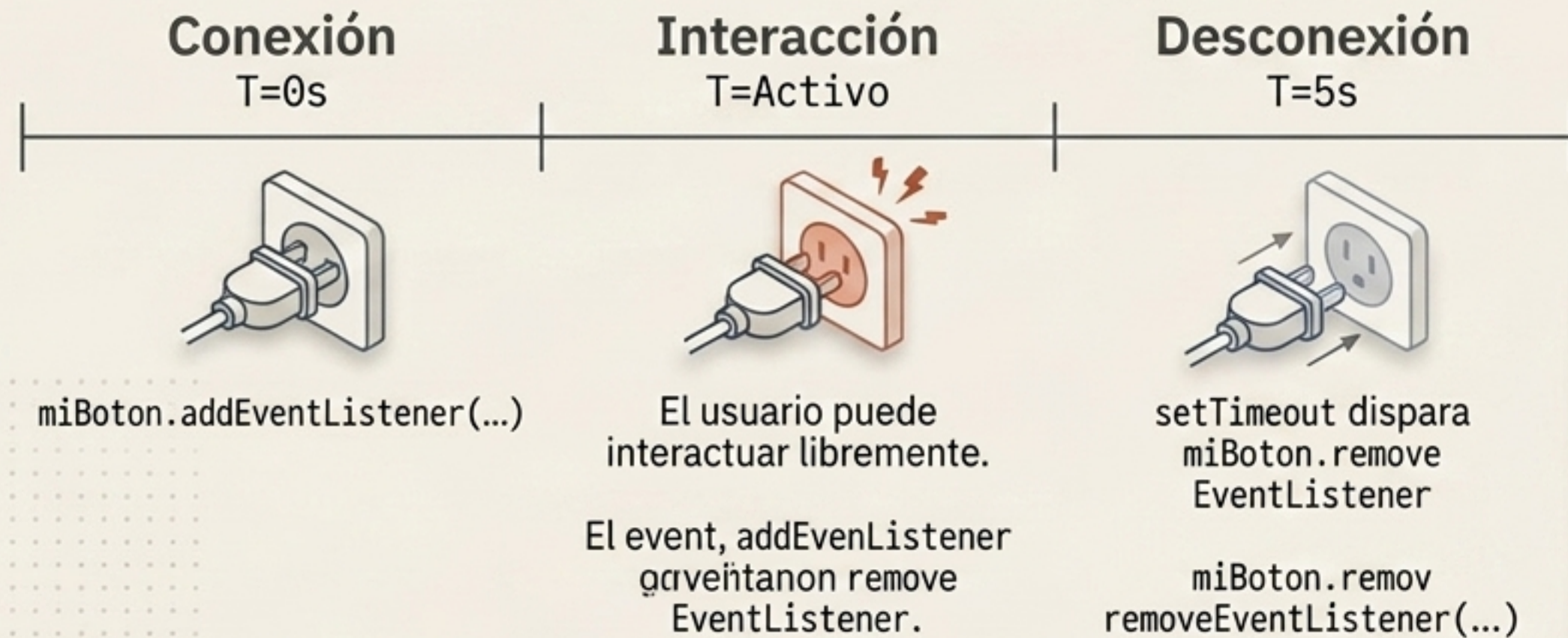


```
addEventListener('click', accion);
```

- **Reutilizable** y mantiene el código estructurado.
- **Obligatoria** si deseas tener la capacidad de desconectar el evento.

# El Ciclo de Vida del Evento

Los oyentes no tienen que ser permanentes. Para optimizar recursos, usamos `removeEventListener` proveyendo el mismo evento y la misma función nombrada.



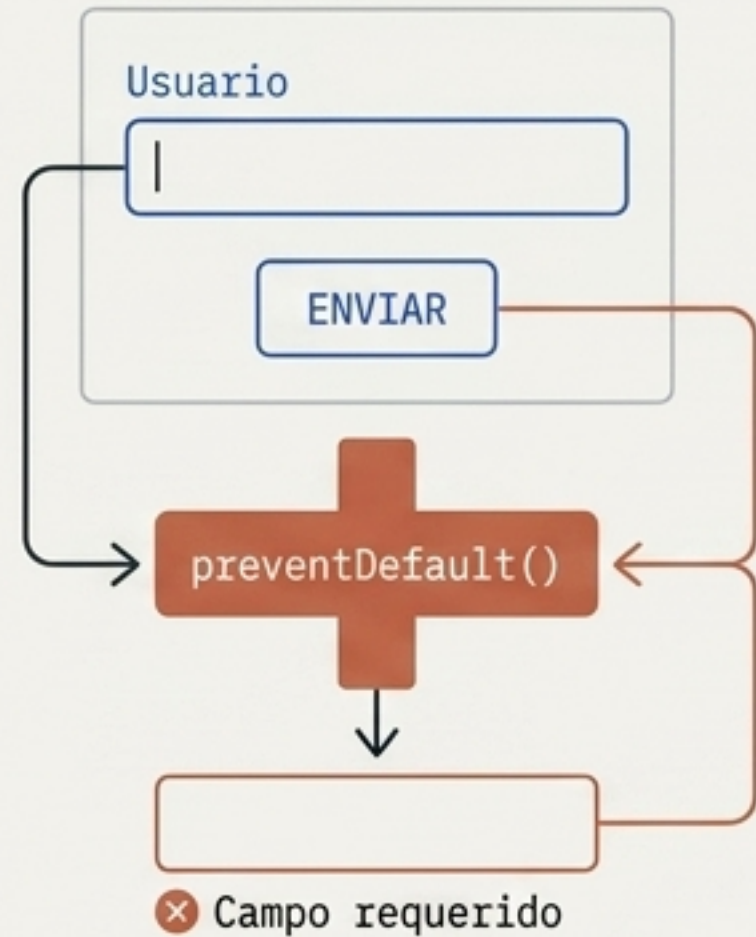
## Implementación con setTimeout

```
function manejador() {  
  console.log('¡Click!');  
}  
  
const miBoton = document.getElementById('miBoton');  
  
// 1. Conectar el oyente  
miBoton.addEventListener('click', manejador);  
  
// 2. Programar la desconexión después de 5 segundos  
setTimeout(() => {  
  miBoton.removeEventListener('click', manejador);  
  console.log('Oyente desconectado.');
```

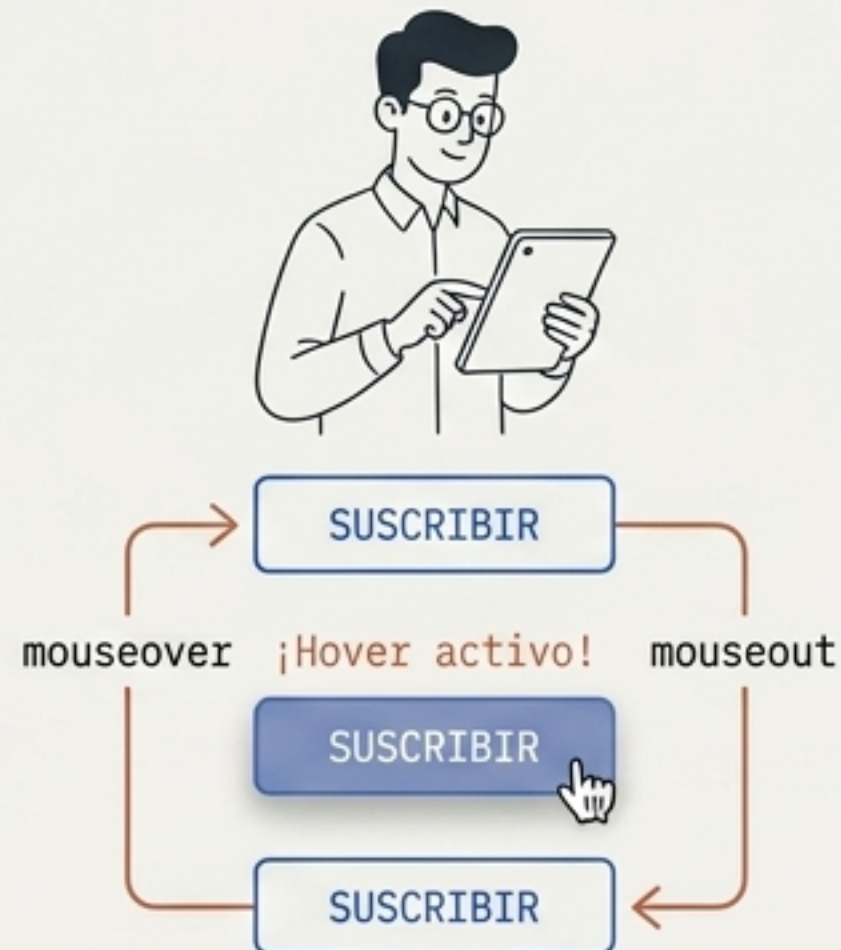
# ¿Para qué usar los eventos?

El control de eventos es el núcleo de la experiencia de usuario (UX) moderna. Permite responder a las necesidades del usuario en tiempo real.

## Validación de Formularios



## Interacción de UI



## Actualización Dinámica



# El Circuito Interactivo Completo

Tu página web ya no es un documento estático; es una aplicación viva.  
Dominar JavaScript es dominar este flujo de información.

